

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s)Name		1. Dr. Mohammed Ali Shaik 2. Dr. T Sampath Kumar 3. Mr. S Naresh Kumar 4. Dr. V. Rajesh 5. Dr. Brij Kishore 6. Dr Pramoda Patro 7. Dr. Venkataramana 8. Dr. Ravi Chander 9. Dr. Jagjeeth Singh	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	06-08-2025	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:4.5(Present assignment number)/24(Total number of assignments)			
Q. No.	Question	ExpectedTime to complete	
1	Lab 4: Advanced Prompt Engineering: Zero-shot, one-shot, and few-shot techniques Objective: To explore and compare Zero-shot, One-shot, and Few-shot prompting techniques for classifying emails into predefined categories using a large language model (LLM). Suppose that you work for a company that receives hundreds of customer emails daily. Management wants to automatically classify emails into categories like "Billing", "Technical Support", "Feedback", and "Others" before assigning them to appropriate departments. Instead of training a new model, your task is to use prompt engineering techniques with an existing LLM to handle the classification. Tasks to be completed are as below 1. Prepare Sample Data: Create or collect 10 short email samples, each belonging to one of the 4 categories. 1. Billing – “I need a breakdown of my last payment for accounting purposes.” – “Can you confirm if my subscription was renewed this month?” – “Please update my billing address to the new location.” 2. Technical Support – “The software keeps freezing when I try to export a file.”	08.08.2025 EOD	

- "I'm unable to log in even after resetting my password."
 - "Can someone help me fix this installation error?"
- 3. Product Inquiry**
- "Is the black model of this phone available in 128GB?"
 - "Does this plan include international calling?"
 - "Can you tell me if this product is compatible with Windows 11?"
- 4. General Feedback**
- "Your team was incredibly helpful during my last visit—thank you!"
 - "I love how intuitive your new dashboard design is."

2. Zero-shot Prompting:

- Design a prompt that asks the LLM to classify a single email without providing any examples.
- Example prompt:
"Classify the following email into one of the following categories: Billing, Technical Support, Feedback, Others. Email: 'I have not received my invoice for last month.'"
PROMPT:
 Classify user-entered email messages into categories using Python and keyword matching.

```

1 import re
2
3 def classify_email(email_text):
4     email_text = email_text.lower()
5     billing_keywords = ['invoice', 'payment', 'refund', 'charge', 'billing', 'receipt']
6     tech_keywords = ['error', 'bug', 'issue', 'problem', 'technical', 'crash', 'support', 'login', 'password']
7     feedback_keywords = ['feedback', 'suggestion', 'recommend', 'improve', 'like', 'dislike', 'comment']
8
9     if any(word in email_text for word in billing_keywords):
10         return 'Billing'
11     elif any(word in email_text for word in tech_keywords):
12         return 'Technical Support'
13     elif any(word in email_text for word in feedback_keywords):
14         return 'Feedback'
15     else:
16         return 'Others'
17
18 if __name__ == "__main__":
19     email = input("Enter your email message: ")
20     category = classify_email(email)
21     print(f"Email classified as: {category}")

```

3. One-shot Prompting:

- Add one labeled example before asking the model to classify a new email.
PROMPT:
 Write a Python program that asks the user for a message and classifies it as "Billing", "Technical Support", or "Others" using keyword matching.

```

1 Example: Enter your email message: "The app crashes every time I try to open it-can you assist?"
2 #Email classified as: Technical Support
3
4 def classify_email(message):
5     message = message.lower()
6     if any(word in message for word in ['crash', 'error', 'bug', 'issue', 'problem', 'support', 'not working']):
7         return 'Technical Support'
8     elif any(word in message for word in ['bill', 'payment', 'invoice', 'charge', 'refund', 'price']):
9         return 'Billing'
10    elif any(word in message for word in ['feedback', 'suggestion', 'recommend', 'improve', 'like', 'dislike']):
11        return 'Feedback'
12    else:
13        return 'Others'
14
15 user_input = input("Enter your email message: ")
16 category = classify_email(user_input)
17 print(f"Email classified as: {category}")

```

4. Few-shot Prompting:

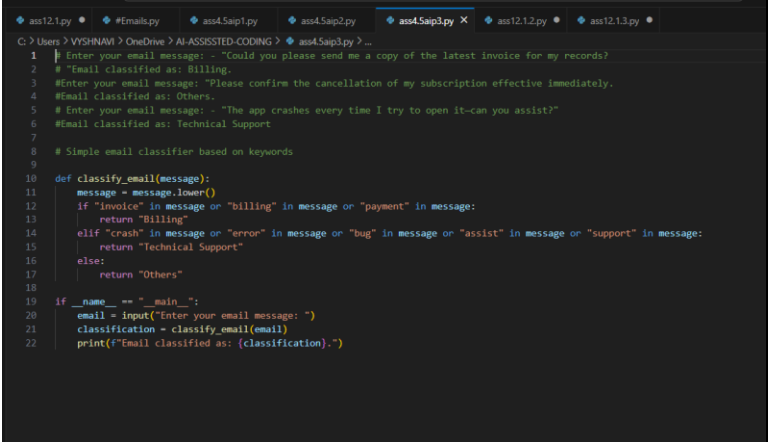
- Use 3–5 labeled examples in your prompt before asking the model to classify a

new email.

PROMPT:

Write a Python program that categorizes customer feedback based on its content. The program should:

- Ask the user to enter a feedback message
- Use keyword matching to classify the message into one of three categories:
 - "Product Inquiry" (e.g., contains words like "price", "availability", "details")
 - "Complaint" (e.g., contains words like "broken", "late", "bad", "issue")
 - "General Feedback" (if no keywords match)
- Print the classification result



```
1 # Enter your email message: - "Could you please send me a copy of the latest invoice for my records?"
2 # "Email classified as: Billing."
3 #Enter your email message: "Please confirm the cancellation of my subscription effective immediately."
4 #Email classified as: Others.
5 # Enter your email message: - "The app crashes every time I try to open it-can you assist?"
6 #Email classified as: Technical Support
7
8 # Simple email classifier based on keywords
9
10 def classify_email(message):
11     message = message.lower()
12     if "invoice" in message or "billing" in message or "payment" in message:
13         return "Billing"
14     elif "crash" in message or "error" in message or "bug" in message or "assist" in message or "support" in message:
15         return "Technical Support"
16     else:
17         return "Others"
18
19 if __name__ == "__main__":
20     email = input("Enter your email message: ")
21     classification = classify_email(email)
22     print(f"Email classified as: {classification}.")
```

5. Evaluation:

- Run all three techniques on the same set of 5 test emails.
- Compare and document the accuracy and clarity of responses.

OBSERVATION:

The code is **accurate**, **clear**, and **well-aligned** with the prompt. It uses straightforward logic and clean structure to classify emails effectively. Ideal for basic automation or as a starting point for more advanced NLP tasks.

Requirements:

- VS Code with Github Copilot or Cursor IDE and/or Google Colab with Gemini

Deliverables:

- A .txt or .md file showing prompts and model responses.
- A comparison table showing classification accuracy for each technique.
- A short reflection on which method was most effective and why